

# Shortest Common Supersequence (LeetCode 1092)

---

## Problem Statement

Given two strings *str1* and *str2*, find the shortest string that contains both strings as subsequences. If multiple answers exist, return any one.

## What is a Supersequence?

A supersequence contains every character of the original string in the same order.

### Example

```
str1 = "abc"
```

### Supersequence:

```
abc  
axbc  
abxc  
axbxc
```

### Not Supersequence:

```
acb  
bac
```

because order changes.

## Observation

Suppose

```
str1 = abac  
str2 = cab
```

If we simply concatenate

```
abac + cab = abaccab
```

Length = 7

But notice

```
abac
  ||
  ab
cab
```

The characters

```
ab
```

are common.

Instead of writing them twice, we write them once.

So

```
cabac
```

Length = 5

This is exactly what LCS tells us.

## Core Intuition

Think of the two strings as railway tracks.

```
str1 : a ---- b ---- a ---- c
str2 : c ---- a ---- b
```

Whenever both strings have the same character,

```
a
b
```

we merge the tracks.

Whenever they differ,

we keep both characters.

So we first find

Longest Common Subsequence

and then merge the remaining characters.

## DP Idea

We first compute

$dp[i][j]$  = Length of LCS of  $str1[0...i-1]$  and  $str2[0...j-1]$

## Transition

If characters match

$$dp[i][j] = 1 + dp[i-1][j-1]$$

Else

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

## DP Table Construction

Example

```
str1 = abac  
str2 = cab
```

Rows = str1

Columns = str2

|   | c | a | b |
|---|---|---|---|
|   | 0 | 0 | 0 |
| a | 0 | 1 | 1 |
| b | 0 | 1 | 2 |
| a | 0 | 1 | 2 |
| c | 1 | 1 | 2 |

More clearly (including headers):

|   | Ø | c | a | b |
|---|---|---|---|---|
| Ø | 0 | 0 | 0 | 0 |
| a | 0 | 0 | 1 | 1 |
| b | 0 | 0 | 1 | 2 |

|          |   |   |   |   |
|----------|---|---|---|---|
| <b>a</b> | 0 | 0 | 1 | 2 |
| <b>c</b> | 0 | 1 | 1 | 2 |

Final LCS length

2

## Reconstruction (Most Important Part)

Now we start from  $(i,j) = (4,3)$  Bottom-right corner.

### Step 1

|   | ∅ | c | a | b   |
|---|---|---|---|-----|
| ∅ | 0 | 0 | 0 | 0   |
| a | 0 | 0 | 1 | 1   |
| b | 0 | 0 | 1 | 2   |
| a | 0 | 0 | 1 | 2   |
| c | 0 | 1 | 1 | 2 ↑ |

Current characters

$str1[3] = c$

$str2[2] = b$

Different.

Compare

Top =  $dp[3][3] = 2$

Left =  $dp[4][2] = 1$

Since  $Top > Left$

Move UP.

Add

c

Result

c

Step 2

Now  $(i,j) = (3,3)$

|   | ∅ | c | a | b   |
|---|---|---|---|-----|
| ∅ | 0 | 0 | 0 | 0   |
| a | 0 | 0 | 1 | 1   |
| b | 0 | 0 | 1 | 2   |
| a | 0 | 0 | 1 | 2 ↑ |
| c | 0 | 1 | 1 | 2   |

Characters

a

b

Different.  
Compare

Top = 2

Left = 1

Move UP.  
Add

a

Result

ca

Step 3 (Match)

Now  $(i,j) = (2,3)$

Characters

*b*

*b*

Match.

|   | ∅ | c | a | b   |
|---|---|---|---|-----|
| ∅ | 0 | 0 | 0 | 0   |
| a | 0 | 0 | 1 | 1   |
| b | 0 | 0 | 1 | 2 ↖ |
| a | 0 | 0 | 1 | 2   |
| c | 0 | 1 | 1 | 2   |

Take only one b.  
Move diagonally.



Result

cab

Step 4 (Match)

Now  $(i,j) = (1,2)$

Characters

*a*

*a*

Match.

|  | ∅ | c | a | b |
|--|---|---|---|---|
|--|---|---|---|---|

|          |   |   |     |   |
|----------|---|---|-----|---|
| ∅        | 0 | 0 | 0   | 0 |
| <b>a</b> | 0 | 0 | 1 ↖ | 1 |
| <b>b</b> | 0 | 0 | 1   | 2 |
| <b>a</b> | 0 | 0 | 1   | 2 |
| <b>c</b> | 0 | 1 | 1   | 2 |

Take only one

a

Move diagonally.

Result

caba

## Step 5

Now  $(i,j) = (0,1)$

No characters remain in str1.

Only c remains in str2.

Append

c

Result

cabac

Reverse? Notice that we built the answer from the end while backtracking, so the collected sequence before reversing is:

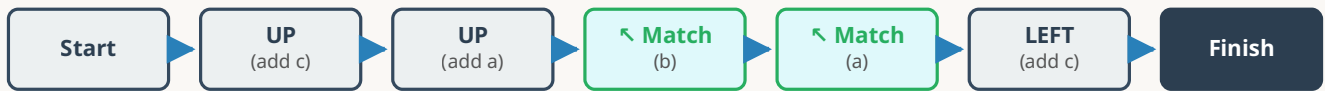
Collected:  $c \rightarrow a \rightarrow b \rightarrow a \rightarrow c$

Reverse it:

cabac

## Complete Backtracking Path

Path



Another way to visualize the movement:

|   | ∅ | c | a | b |
|---|---|---|---|---|
| ∅ | 0 | 0 | 0 | 0 |
| a | 0 | 0 | 1 | 1 |
| b | 0 | 0 | 1 | 2 |
| a | 0 | 0 | 1 | 2 |
| c | 0 | 1 | 1 | 2 |

★ Start

★ = Start

↑ = Move up (take 'c')

↑ = Move up (take 'a')

↖ (green arrows) = Diagonal match (take one common character)

← = Left after reaching top row (take remaining 'c')

## Why do we compare Top and Left?

Suppose characters are different.

```
str1[i-1] != str2[j-1]
```



Then

Top = Ignore current character of str1

Left = Ignore current character of str2

The larger DP value preserves a longer LCS, which means we keep more shared characters. Keeping a longer LCS minimizes duplicate characters, leading to the shortest possible supersequence.

So:

if(top > left)

take str1 character

move UP

otherwise

take str2 character

move LEFT

## Complete Algorithm

1. Compute the LCS DP table.
2. Start from the bottom-right corner.
3. If characters match:
  - Add the character once.
  - Move diagonally.
4. Otherwise:
  - If the top cell has a larger value, add the current character from str1 and move up.
  - Else, add the current character from str2 and move left.
5. When one string finishes, append the remaining characters from the other string.
6. Reverse the built string (because it was constructed backwards).

## Time Complexity

Building DP table:  $O(n \times m)$

Reconstructing the answer:  $O(n + m)$

Overall:

$O(n \times m)$

## Space Complexity

Using the full DP table:

$O(n \times m)$

## Interview takeaway

The problem is not solved by directly finding the SCS. Instead:

- First compute the LCS.
- Use the LCS DP table to backtrack and merge both strings.
- Match → include the character once and move diagonally.
- Mismatch → follow the direction (up or left) with the larger LCS value, adding the corresponding character.

Reverse the constructed string at the end because backtracking builds it from the last character to the first.

This "compute DP first, then reconstruct the answer by backtracking" pattern is a classic dynamic programming technique that also appears in problems like printing the LCS, edit-distance reconstruction, and sequence alignment.